

"گزارش تمرین عملی"

عنوان گزارش: تمرین عملی شماره 1: کاربرد الگوریتم های جست و جوی تپه نوردی و شبیه سازی ذوب فلزات در دنیای حقیقی

شرح مختصری از مسئله: هدف مسئله استفاده از دو الگوریتم جست و جوی محلی شبیه سازی ذوب فلزات و تپه نوردی جهت حل سیستم معادلاتی فرومغین (underdetermined- سیستم معادلاتی که در آن تعداد معادلات کمتر از تعداد مجهولات میباشد) است؛ مسئله ای که حل آن به روش سنتی چالش برانگیز بوده و شامل چندین راه حل محتمل میباشد. برنامه ی طراحی شده میبایست فایل متنی (coefficient.txt) حاوی سیستم معادلات را به عنوان ورودی بخواند و جواب بهینه، مقدار خطا و زمان صرف شده را خروجی دهد.

1Task: شناسایی همسایه های مجاور و ایجاد تابع هزینه (Cost Function)

- ✓ شناسایی همسایه های مجاور: در الگوریتم های جست و جوی محلی همانند جست و جوی تپه نوردی و شبیه سازی ذوب فلزات، همسایه های مجاور به راه حل هایی گفته میشود که به راه حل موجود فعلی نزدیک هستند. برای شناسایی آن ها باید به سه مورد در مسئله توجه کنیم:
1. تعریف فضای راه حل (Solution Space): فضای راه حل شامل تمام مقادیر ممکن برای مجهولات در محدوده ی داده شده است؛ به طور مثال در بازه ی $[-1000, 1000]$.

2. ساختن همسایه ها (Generating Neighbors): ساختن همسایه ها برای یک راه حل معین از طریق ایجاد تغییرات کوچک در مقدار مجهولات است. به فرض اگر یک راه حل معین داشته باشیم، اضافه یا کم کردن دلتا که مقداری اندک (به اندازه ی گام است) به مجهولات، همسایه های راه حل فعلی را میسازد.

3. اندازه ی گام (Step Size): اندازه گام تعیین می کند که به چه میزان مقدار یک مجهول را برای ایجاد یک همسایه تغییر می دهیم. اندازه گام می تواند یک مقدار ثابت بوده و یا به صورت پویا باشد. انتخاب نوع اندازه گام بر اساس الزامات الگوریتم صورت میگیرد.

- نکته 1: از آنجائیکه پویا و یا ثابت بودن اندازه گام در پیاده سازی الگوریتم ها و لذا در نتایج بدست آمده برای مسئله تاثیرگذار است، هر دو نوع اندازه گام بررسی گردید و پس از مقایسه آنها با یکدیگر اندازه گام پویا برای ممانعت از به دام افتادن الگوریتم در اکسترمم محلی، سازگاری بیشتر و کارآمدتر گشتن الگوریتم ها، جهت پیاده سازی در مسئله انتخاب شد.
- اندازه گام ثابت: مزایا < پیاده سازی آن در مقایسه با اندازه گام پویا ساده میباشد به علاوه میتوان گفت جست و جویی یکنواخت از فضای حالت فراهم میکند. / معایب < اگر خیلی بزرگ یا خیلی کوچک در نظر گرفته شده باشد میتواند منجر به گیر افتادن الگوریتم در اکسترمم محلی شود همچنین اگر اندازه در نظر گرفته شده برای گام مناسب نباشد ممکن است به تکرار های زیادی برای یافتن راه حل بهینه سراسری نیاز باشد.

اندازه گام پویا: مزایا < به دلیل سازگاری که با دور نمای فضای حالت دارد مانع از به دام افتادن الگوریتم در اکسترمم های محلی میشود همچنین از آنجائیکه این امکان وجود دارد که در مواقعی که از راه حل بهینه دور است گام های بزرگ و در مواقعی که به آن نزدیک است گام های کوچک بردارد، همگرایی را افزایش میدهد و لذا بسیار کارآمد تر از اندازه گام ثابت عمل میکند. / معایب < مسلما پیاده سازی آن دشوار تر است و به پارامتر های اضافه برای تنظیم تغییرات آن نیاز داریم به علاوه تنظیم آن به طوری که جست و جو در فضای حالت ها به شکلی متعادل و درست انجام شود باید دقیق انجام شود که این امر آسان نیست.

- نکته 2: هنگامی که از اندازه گام پویا برای پیاده سازی الگوریتم ها استفاده میکنیم، این دغدغه وجود دارد که در زمان ساخت همسایه های راه حل فعلی، مقدار مجهولات ساخته شده از

محدوده و بازه ی از پیش تعریف شده در مسئله تجاوز کنند لذا به مکانیسمی جهت پیشگیری از این مشکل نیاز داریم. چند استراتژی برای این منظور وجود دارد (سه مورد از آنها را در ادامه بررسی کرده ام) که پس از مقایسه، مکانیسم Clipping به سبب اجرای آسانتر برای پیاده سازی الگوریتم ها انتخاب شد.

Clipping: ابتدا مجهولات را با اندازه گام پویا به روزرسانی میکنیم و سپس بررسی میکنیم آیا مقدار آن ها خارج از بازه مسئله است یا خیر. در صورتی که مجهولی مقداری خارج از محدوده داشت آن را به نزدیک ترین مقدار مرزی کوتاه (یا همان برش و Clip) میکنیم. این کار به کمک توابع Max و Min انجام میشود.

Adaptive Step Size: یک استراتژی دیگر استفاده از اندازه گام پویایی است که قدرت تطبیق با شرایط را دارد و بر اساس نزدیکی به مرزها تعیین میشود. به طور مثال اگر مقدار مجهول در حالت فعلی به مرز مسئله نزدیک است، اندازه گام پویا کاهش میابد تا از خارج شدن آن پس از بروزرسانی از مرز جلوگیری کند.

Reflective Boundaries: در این استراتژی به جای برش دادن، مقادیر را به داخل محدوده مسئله بازتاب میدهیم. به فرض مقدار مجهول پس از بروزرسانی با اندازه گام پویا بیشتر از 1000 شد در این صورت مقدار آن را روی 1000 منهای مقدار مازاد بر 1000 تنظیم میکنیم و یا اگر مقدار مجهول پس از بروزرسانی کمتر از 1000- گردید، مقدار آن را بر روی 1000- به علاوه ی مقدار کسری تا 1000- قرار میدهیم.

- نکته 3: گفتیم جهت تعیین فضای راه حل به دنبال مقادیر ممکن برای مجهولات در بازه مسئله هستیم، با توجه به دو نکته ی قبل در این مسئله ابتدا لازم است مقدار دهی اولیه کنیم (Initialization). به عبارتی از یک راه حل اولیه شروع کنیم که در آن به هر مجهول یک مقدار اولیه به کمک random.uniform در محدوده و بازه مسئله داده شده است. سپس با ایجاد کمی تغییر در مجهولات و بروزرسانی آنها (یعنی استفاده از اندازه گام پویا) همسایه ها را بسازیم و در آخر (به کمک استراتژی Clipping) از داخل بازه ماندن مجهولات پس از بروزرسانی اطمینان حاصل کنیم.

✓ ایجاد تابع هزینه (Cost Function):

تابع هزینه یا تابع هدف (Objective Function) میزان صدق یک راه حل داده شده در سیستم معادلات را اندازه گیری میکند. { میدانیم که در الگوریتم های جست و جوی محلی در بررسی State-Space Landscape محور افقی State-Space یا همان فضای حالت است و محور عمودی Elevation یا ارتفاع است که بسته به نوع مسئله میتواند متناظر با Objective Function یا همان تابع هدف باشد که در این صورت ما به دنبال نقطه بهینه سراسری یا همان Global Maxima هستیم؛ و در صورتی که Elevation متناظر با Cost Function یا همان تابع هزینه باشد در واقع نوع مسئله به شکلی است که به دنبال نقطه مینیمم سراسری یا همان Global Minima میگردیم.}

توصیف و تشکیل تابع هزینه در مسئله ی ما سه مرحله دارد:

1. محاسبه ی خطا (Error Calculation): اگر برای هر معادله تفاوت سمت چپ و راست را محاسبه کنیم یعنی به عبارتی به کمک مقدار فعلی مجهولات مابه التفاوت Left Hand Side LHS و Right Hand Side- RHS را بدست آوریم؛ خطای آن معادله را محاسبه کرده ایم.

2. محاسبه ی مجموع مربعات خطا ها (Sum of Squared Errors): برای بدست آوردن یک مقدار واحد که نشان دهنده ی هزینه باشد؛ مجذور خطای هر معادله را محاسبه کرده و مقادیر بدست آمده از تمامی معادلات سیستم را با هم جمع میکنیم. با این کار خطا های مثبت و منفی با هم خنثی نمیشوند بلکه بر بزرگ بودن خطا تاکید میکنند.

3. به حداقل رساندن هزینه (Minimizing the Cost): حال هدف الگوریتم بهینه سازی یافتن مجهولاتی است که تابع هزینه را به حداقل برساند.

Task 2: پیاده سازی دو الگوریتم جست و جوی تپه نوردی غیر قطعی و تپه نوردی با شروع مجدد تصادفی، نقاط شروع اولیه، تعداد تکرار ها و بررسی پارامتر ها

✓ الگوریتم جست و جوی تپه نوردی غیر قطعی (Stochastic Hill Climbing Search):
توصیف مراحل الگوریتم:

- به مجهولات مقدار اولیه ای تصادفی و رندم (راه حلی در بازه مورد قبول مسئله) میدهیم.
- با تنظیم کردن یک مجهول، همسایه تصادفی میسازیم.

- از یک اندازه گام پویای مناسب در محدوده مسئله استفاده میکنیم.
- اگر همسایه هزینه کمتری داشته باشد، راه حل را بروزرسانی میکنیم.

نقاط شروع اولیه (Starting Points):

- مقدار دهی اولیه: الگوریتم با یک راه حل اولیه تصادفی در محدوده مشخص شده شروع می شود. این کار با استفاده از تابع `random.uniform(range_min, range_max)` برای هر مجهول انجام میشود.
- تصادفی بودن: تصادفی بودن در نقطه شروع اولیه به جست و جو کردن در مناطق مختلف فضای راه حل کمک می کند، که این موضوع برای اجتناب از گیر افتادن الگوریتم در بهینه محلی بسیار مهم است.

تعداد تکرارها (Number of Iterations):

- حداکثر تکرار: الگوریتم برای تعداد ثابتی از تکرارها اجرا می شود که توسط پارامتر `max_iterations` مشخص شده است. در هر تکرار، یک همسایه ایجاد میشود و بر اساس تابع هزینه تصمیم گرفته میشود که به این همسایه منتقل شود یا خیر.
- اندازه گام پویا: اندازه گام به صورت پویا در طول تکرارها تنظیم می شود تا جست و جو و بهره برداری متعادل شود. اگر همسایه بهتری پیدا شود، اندازه گام افزایش می یابد. در غیر این صورت کاهش پیدا میکند.

پارامترها (Parameters):

1. Initial Step Size `(initial_step_size)`:

- مقدار فعلی: 0.1
- تنظیم: مقادیر مختلفی مانند 0.1 و 0.5 و 2.0 و غیره را امتحان میکنیم. گام هایی با اندازه ی کوچکتر می تواند به تنظیمات دقیق تری منجر شود، در حالی که اندازه های بزرگتر می توانند فضای راه حل را به طور گسترده تری بررسی کنند.

2. Maximum Iterations `(max_iterations)`:

- مقدار فعلی: 10000
- تنظیم: تعداد تکرارها را به 20000 و 50000 یا بیشتر افزایش میدهیم تا به الگوریتم زمان بیشتری برای جست و جوی فضای راه حل داده شود.

✓ الگوریتم جست و جوی تپه نوردی با شروع مجدد تصادفی (Random Restart Hill Climbing):
توصیف مراحل الگوریتم:

- از الگوریتم جست و جوی تپه نوردی با چندین شروع و راه اندازی مجدد استفاده میکنیم.
- هر راه اندازی مجدد با یک راه حل اولیه تصادفی جدید شروع می شود.
- الگوریتم بهترین راه حل پیدا شده در تمام راه اندازی های مجدد را پیگیری میکند.

نقاط شروع اولیه (Starting Points):

- راه اندازی مجدد چندگانه: این الگوریتم چندین جستوجوی تپه نوردی را هر بار با راه حل اولیه تصادفی متفاوتی، اجرا میکند. این کار با فراخواندن متعدد تابع hill_climbing هر بار از راه حل اولیه ی متفاوتی اتفاق می افتد.
- تنوع: با راه اندازی های مجدد که نقاط اولیه مختلفی دارند، الگوریتم شانس پیدا کردن نقاط سراسری بهتر را افزایش میدهد چراکه قسمت های مختلفی از فضای حالت را بررسی میکند.

تعداد تکرارها (Number of Iterations):

- حداکثر تکرار در هر راه اندازی مجدد: مشابه با الگوریتم تپه نوردی غیرقطعی، هر اجرا از الگوریتم تپه نوردی در چارچوب شروع تصادفی یا Random Restart برای تعداد ثابتی از تکرارها (max_iterations) اجرا می شود.
- تعداد راه اندازی مجدد: تعداد کل راه اندازی های مجدد توسط پارامتر راه اندازی مجدد مشخص می شود. هر راه اندازی مجدد با یک راه حل اولیه تصادفی جدید شروع می شود و الگوریتم تپه نوردی را برای تعداد تکرار مشخصی اجرا می کند.

- ردیابی بهترین راه حل: پس از هر راه اندازی مجدد، الگوریتم راه حل یافت شده را با بهترین راه حل تا همان لحظه مقایسه می کند و در صورت بهتر بودن راه حل جدید، بهترین راه حل را به روزرسانی میکند.

پارامترها (Parameters):

1. Initial Step Size (initial_step_size):

- مقدار فعلی: 0.1
- تنظیم: مقادیر مختلف مانند 0.1 و 0.5 و 2.0 و غیره را امتحان میکنیم.

2. Maximum Iterations per Restart (max_iterations):

- مقدار فعلی: 1000
- تنظیم: تعداد تکرارها در هر راه اندازی مجدد را به 2000 و 5000 و یا بیشتر افزایش میدهم.

3. Number of Restarts (restarts):

- مقدار فعلی: 10
- تنظیم: برای جست و جوی نقاط شروع اولیه، تعداد راه اندازی های مجدد را به 20 و 50 و یا بیشتر افزایش میدهم.

Task 3: پیاده سازی الگوریتم شبیه سازی ذوب فلزات (Simulated Annealing) و بررسی

پارامترها

توصیف مراحل الگوریتم:

- مقدار دهی اولیه (Initialization): الگوریتم با یک راه حل اولیه تصادفی در محدوده مشخص شده مسئله شروع می شود. دمای اولیه و اندازه گام نیز تنظیم میشود.
- تابع هزینه (Cost Function): تابع هزینه مجموع مجذور خطاها را برای سیستم معادلات محاسبه می کند.
- ساخت همسایه (Generating Neighbor): یک همسایه تصادفی با تنظیم یک مجهول توسط یک مقدار تصادفی در محدوده ی اندازه گام ایجاد می شود.

- احتمال (Probability): اگر همسایه هزینه کمتری داشته باشد پذیرفته می شود اما اگر همسایه هزینه بیشتری داشته باشد با احتمالی که با دما کاهش می یابد پذیرفته می شود.
- برنامه خنک کننده (Cooling Schedule): درجه حرارت به تدریج با توجه به سرعت خنک کننده کاهش می یابد. اندازه گام به صورت پویا تنظیم می شود تا جست و جو و بهره برداری را متعادل کند.

پارامترها (Parameters):

1. Initial Step Size (initial_step_size):

- مقدار فعلی: 1.0
- تنظیم: مقادیر مختلف مانند 0.1 و 0.5 و 2.0 را امتحان میکنیم.

2. Initial Temperature (initial_temp):

- مقدار فعلی: 1000.0
- تنظیم: با دماهای اولیه مختلف مانند 500.0 و 2000.0 و غیره آزمایش میکنیم.
- دمای اولیه بالاتر در ابتدا امکان جست و جوی بیشتر را فراهم میکند.

3. Cooling Rate (cooling_rate):

- مقدار فعلی: 0.995
- تنظیم: نرخ های خنک کننده متفاوتی مانند 0.99 و 0.999 و غیره را امتحان میکنیم. سرعت خنک کننده کمتر (نزدیک به 1.0) امکان جست و جوی بیشتر را در مدت زمان طولانی تری فراهم میکند.

4. Maximum Iterations (max_iterations):

- مقدار فعلی: 10000
- تنظیم: تعداد تکرارها را به 20000 و 50000 و یا بیشتر افزایش میدهم.

4Task: تجزیه و تحلیل و بحث در مورد نتایج، تجارب و بینش و نتیجه گیری

✓ تجزیه و تحلیل و بحث در مورد نتایج:

❖ الگوریتم جست و جوی تپه نوردی غیر قطعی (Stochastic Hill Climbing)

➤ نتایج

- دقت: الگوریتم می تواند راه حل نسبتاً خوبی را پیدا کند، اما ممکن است به دلیل ماهیت حریصانه ای که دارد در نقطه بهینه محلی گیر کند.
- کارایی: نسبتاً سریع است، به خصوص اگر اندازه گام و تعداد تکرار آن به خوبی تنظیم شده باشد.
- سازگاری: نتایج بسته به نقطه شروع اولیه و اندازه گامشان میتوانند به طور قابل توجهی متفاوت باشند.

➤ ویژگی ها

- سادگی: پیاده سازی و درک این الگوریتم آسان است.
- جستجوی محلی: الگوریتم بر جست و جو در همسایگی راه حل فعلی تمرکز می کند.
- اندازه گام پویا: تنظیم اندازه گام به صورت پویا به تعادل جست و جو و بهره برداری کمک می کند.

➤ یافته ها

- بهینه محلی: الگوریتم ممکن است با بهینه محلی، به ویژه در فضاهای حل پیچیده مشکل داشته باشد.
- حساسیت پارامتر: عملکرد به انتخاب اندازه گام و تعداد تکرار بسیار حساس است.

❖ الگوریتم جست و جوی تپه نوردی با شروع مجدد تصادفی (Random Restart Hill Climbing)

➤ نتایج

- دقت: عموماً به دلیل راه اندازی مجدد چند گانه ای که دارد، راه حل های بهتری نسبت به تپه نوردی تصادفی پیدا می کند.
- کارایی: به دلیل راه اندازی مجدد اضافی کمی کند تر از تپه نوردی تصادفی است، اما همچنان کارآمد است.
- سازگاری: با بررسی چندین نقطه شروع اولیه، نتایج منسجم تری دارد.

➤ ویژگی ها

- راه اندازی مجدد چند گانه: با شروع از نقاط مختلف در فضای راه حل به فرار از بهینه محلی کمک می کند.
- اندازه گام پویا: مشابه با تپه نوردی تصادفی، تنظیم اندازه گام به صورت پویا عملکرد را بهبود می بخشد.

➤ یافته ها

- جستجوی سراسری: در یافتن بهینه سراسری در مقایسه با یک بار تپه نوردی مؤثرتر است.
- رابطه ی دو طرفه: بین تعداد راه اندازی مجدد و زمان محاسباتی تعادل وجود دارد.

❖ الگوریتم شبیه سازی ذوب فلزات (Simulated Annealing)

➤ نتایج

- دقت: به دلیل پذیرش احتمالی راه حل های بدتر، اغلب بهترین راه حل ها را در بین سه الگوریتم پیدا می کند.

○ راندمان: به دلیل برنامه خنک کننده و نیاز به تکرارهای زیاد، می تواند کندتر باشد.

○ سازگاری: نتایج منسجم تری را ارائه می دهد زیرا فضای راه حل را به طور کامل جست و جو می کند.

➤ ویژگی ها

○ پذیرش احتمالی: راه حل های بدتر احتمال مشخصی دارند ولذا به فرار از بهینه محلی کمک می کند.

○ برنامه خنک کننده: به تدریج دما را کاهش می دهد تا جست و جو و بهره برداری متعادل شود.

○ اندازه گام پویا: تنظیم اندازه گام به صورت پویا روند جستجو را بیشتر بهبود میبخشد.

➤ یافته ها

○ بهینه سراسری: به دلیل توانایی آن در فرار از بهینه محلی، احتمال بیشتری برای یافتن بهینه سراسری دارد.

○ تنظیم پارامتر: به تنظیم دقیق پارامترهایی مانند دمای اولیه، سرعت خنک کننده و اندازه گام نیازمند است.

✓ بینش و تجارب:

❖ الگوریتم جست و جوی تپه نوردی غیر قطعی (Stochastic Hill Climbing)

▪ نقاط قوت: ساده و سریع عمل میکند، برای مشکلاتی که بهینه محلی نگرانی اصلی نیست، خوب است.

▪ نقاط ضعف: مستعد گیر افتادن در بهینه محلی و حساس به پارامترهای اولیه است.

❖ الگوریتم جست و جوی تپه نوردی با شروع مجدد تصادفی (Random Restart Hill Climbing)

- نقاط قوت: به دلیل راه اندازی مجدد چند گانه، در یافتن بهینه سراسری بهتر و قوی تر از تپه نوردی غیر قطعی عمل میکند.
- نقاط ضعف: افزایش زمان محاسباتی به دلیل راه اندازی مجدد چند گانه و همچنان به انتخاب پارامتر حساس است.

❖ الگوریتم شبیه سازی ذوب فلزات (Simulated Annealing)

- نقاط قوت: در فرار از بهینه محلی عالی است، در فضای راه حل جست و جوی کاملتر دارد، اغلب بهترین راه حل ها را پیدا می کند.
- نقاط ضعف: عملکرد کندتر به دلیل برنامه خنک کننده و نیاز به تنظیم دقیق پارامتر دارد.

✓ نتیجه گیری:

هر یک از این سه الگوریتم نقاط قوت و ضعف خود را دارند و در کل انتخاب الگوریتم به مسئله و الزامات خاص آن وابسته است. برای مسئله ی سیستم معادلاتی (underdetermined- سیستم معادلاتی که در آن تعداد معادلات کمتر از تعداد مجهولات میباشد) ، الگوریتم Simulated Annealing به علت توانایی بالایی که در فرار از بهینه محلی و جست و جوی کامل فضای راه حل دارد به طور کلی بهترین نتایج را ارائه می دهد. با این حال، الگوریتم Random Restart Hill Climbing نیز یک رقیب قوی است که تعادل خوبی بین جست و جو و کارایی محاسباتی برای این مسئله ارائه می کند.